

# ORGNZ-03: Bucket Strategy and Design

**Series:** ORGNZ | **Notebook:** 3 of 9 | **Created:** January 2026 | **Last Updated:** 01/28/2026

## Overview

A well-defined bucket strategy optimizes query performance, controls costs, and ensures compliance. This notebook covers naming conventions, retention planning, organizational alignment, and common bucket design patterns.

## Prerequisites

| Requirement           | Details                                                           |
|-----------------------|-------------------------------------------------------------------|
| Dynatrace Environment | SaaS environment with Grail enabled                               |
| Permissions           | <code>storage:bucket-definitions:write</code> for bucket creation |
| Knowledge             | Completed ORGNZ-02                                                |

## Learning Objectives

- By the end of this notebook, you will:
- Design effective bucket naming conventions
  - Plan retention strategies for different use cases
  - Understand cost implications of bucket design
  - Apply organizational alignment patterns

# Bucket Naming Conventions

## Recommended Naming Patterns

| Pattern        | Format                          | Example            | Use Case                          |
|----------------|---------------------------------|--------------------|-----------------------------------|
| Provider-based | <provider>_<type>_<retention>   | aws_logs_35d       | Multi-cloud environments          |
| LOB-based      | <provider>_<type>_<lob>         | aws_logs_finance   | Line of business cost attribution |
| Team-based     | team_<name>_<type>              | team_platform_logs | Team ownership and billing        |
| Compliance     | <regulation>_<type>_<retention> | hipaa_logs_7y      | Regulatory requirements           |

## Detailed Examples

| Bucket Name                | Provider | Data Type | Retention | Org Unit         |
|----------------------------|----------|-----------|-----------|------------------|
| aws_logs_35d_platform      | AWS      | logs      | 35 days   | Platform team    |
| azure_metrics_90d_finance  | Azure    | metrics   | 90 days   | Finance          |
| gcp_spans_14d_checkout     | GCP      | spans     | 14 days   | Checkout service |
| audit_logs_365d_compliance | Any      | logs      | 365 days  | Compliance       |
| debug_logs_7d              | Any      | logs      | 7 days    | Development      |

## Naming Rules Reminder

| Rule               | Requirement                                  |
|--------------------|----------------------------------------------|
| First character    | Must be a letter                             |
| Allowed characters | Lowercase alphanumeric, underscores, hyphens |
| Case               | Lowercase only                               |
| Reserved names     | Cannot use <code>default_*</code> prefix     |
| Immutability       | Names cannot be changed after creation       |

## Retention Strategy

---

### Retention Period Guidelines

| Use Case             | Recommended Retention | Rationale                               |
|----------------------|-----------------------|-----------------------------------------|
| Debug/Development    | 3-7 days              | Short-term troubleshooting, high volume |
| Standard Operations  | 35 days               | Monthly trends, incident response       |
| Performance Analysis | 60-90 days            | Quarterly comparisons                   |
| Compliance/Audit     | 365-3657 days         | Regulatory requirements                 |

## Retention by Data Type

| Data Type        | Typical Retention | Notes                            |
|------------------|-------------------|----------------------------------|
| Debug logs       | 3-7 days          | High volume, low long-term value |
| Application logs | 35-90 days        | Operational analysis             |
| Audit logs       | 1-7 years         | Compliance requirements          |
| Metrics          | 35-90 days        | Trending and alerting            |
| Spans            | 10-35 days        | APM troubleshooting              |
| Security events  | 90-365 days       | Security investigations          |

## Bucket Design Patterns

### Pattern 1: Long-Term Compliance Data

Store critical audit information for extended periods:

```
Bucket: compliance_audit_logs
Table: logs
Retention: 1825 days (5 years)
Use case: SOX compliance, security audits
Route via: OpenPipeline filter on audit-level logs
```

### Pattern 2: Short-Term Debug Logs

Reduce costs by storing verbose logs briefly:

```
Bucket: debug_logs_7d
Table: logs
Retention: 7 days
Use case: Development troubleshooting
Route via: OpenPipeline filter on loglevel=DEBUG
```

### Pattern 3: Team Isolation

Separate buckets for team-based access and cost attribution:

Bucket: team\_platform\_logs  
Table: logs  
Retention: 35 days  
Use case: Platform team owns infrastructure logs  
Access: IAM policy restricts to platform team

### Pattern 4: Application-Specific Retention

Business units with specific retention needs:

Bucket: finance\_app\_logs  
Table: logs  
Retention: 180 days  
Use case: Financial application compliance  
Cost: Chargeback to Finance cost center

## Cost Control and Attribution

### Cost Factors

| Factor                  | Impact on Cost                               |
|-------------------------|----------------------------------------------|
| Ingest volume (GiB/day) | Direct relationship                          |
| Retention period        | Storage costs increase with longer retention |
| Query frequency         | Query costs accumulate                       |

### Cost Attribution Strategy

| Organization Level | Bucket Strategy                     |
|--------------------|-------------------------------------|
| Business Unit      | One bucket per BU                   |
| Cost Center        | Map buckets to cost centers         |
| Cloud Account      | Separate by AWS/Azure/GCP account   |
| Application        | Critical apps get dedicated buckets |

# Analyzing Bucket Usage

```
// Analyze log volume by bucket over last 24 hours
fetch logs
| summarize recordCount = count(), by:{dt.system.bucket}
| sort recordCount desc
```

```
// Track log ingest trends by bucket over time
fetch logs
| summarize recordCount = count(), by:{time_bucket = bin(timestamp, 1h),
dt.system.bucket}
| sort time_bucket desc
```

```
// Compare bucket retention configurations
fetch dt.system.buckets
| fields bucket.name, bucket.table, bucket.retentionDays
| sort bucket.retentionDays desc
```

# Bucket Strategy Considerations

## Access Control Options

Buckets can serve as an access control mechanism:

| Access Method                   | Description                           | When to Use                     |
|---------------------------------|---------------------------------------|---------------------------------|
| Security context (record-level) | Fine-grained ABAC at the record level | Complex multi-team scenarios    |
| Bucket-level IAM policies       | Restrict access to entire buckets     | Simple team isolation           |
| Combined approach               | Security context + bucket policies    | Maximum control and flexibility |

### Bucket-Based Team Isolation Example:

```
Team A: Access to team_a_logs bucket only
Team B: Access to team_b_logs bucket only
Platform: Access to all buckets
```

## Query Performance at Scale

Bucket sizing directly impacts queryability due to the 500 GB scan limit:

| Bucket Ingest | Impact                                |
|---------------|---------------------------------------|
| ~1 TB/day     | Optimal - can query ~12 hours of data |
| 1-3 TB/day    | Acceptable - limited query window     |
| >3 TB/day     | Split bucket or use aggregations      |

## Data Immutability

| Operation                 | Possible? | Alternative                     |
|---------------------------|-----------|---------------------------------|
| Move data between buckets | No        | Route new data via OpenPipeline |
| Selective record deletion | No        | Wait for retention to expire    |
| Bucket deletion           | Yes       | Deletes ALL data permanently    |

## Routing Data to Buckets

Use OpenPipeline to route data to custom buckets:

```
# OpenPipeline routing rule example
processors:
  - type: route
    rules:
      - condition: "loglevel == 'DEBUG'"
        destination: "debug_logs_7d"
      - condition: "log.source contains 'audit'"
        destination: "audit_logs_365d"
      - condition: "host.group starts-with 'finance-'"
        destination: "finance_app_logs"
    default: "default_logs"
```

## Bucket Design Checklist

Use this checklist when planning buckets:

- [ ] Defined naming convention aligned with organization
- [ ] Identified retention requirements by data type
- [ ] Mapped organizational units to buckets for cost attribution
- [ ] Calculated expected ingest volumes (keep <1 TB/day)
- [ ] Planned for compliance requirements
- [ ] Documented bucket purposes
- [ ] Planned OpenPipeline routing rules
- [ ] Considered access control strategy (bucket vs security context)

## Next Steps

---

Continue with the ORGNZ series:

- **ORGNZ-04:** Permissions in Grail Overview

## References

---

- [Grail Buckets](#)
  - [Data Retention](#)
  - [OpenPipeline Routing](#)
- 

*This notebook was AI-generated from Dynatrace documentation and enterprise best practices. It is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*